

OAuth2 Client Authentication Part 1 And Part 2

Introduction :

OAuth2 Client Authentication is a process by which a client application (such as a web app or mobile app) proves its identity to an OAuth2 authorization server when requesting access tokens. It ensures that the client requesting access is authorized to do so, which is crucial for maintaining the security and integrity of the OAuth2 flow.

OAuth2 Client Authentication ensures:

1. **Security:** Only legitimate clients can access resources.
 1. **For example,** it prevents unauthorized apps from posting on your social media account.
2. **Authorization:** Confirms the client app has proper permissions.
 1. **For instance,** it verifies a weather app is allowed to access weather data.
3. **Token Exchange:** Validates that a client can exchange an authorization code for an access token.
 1. **For example,** it ensures a third-party app is legitimate before granting access to a user's profile information.

OAuth2 Client Authentication is used in:

1. **Authorization Code Flow:** When a client exchanges an authorization code for an access token, it must authenticate itself to the authorization server.
 1. **Example:** When you log in, the fitness app gets an authorization code from Google. To turn this code into an access token (which lets the app fetch your profile info), the app must prove it's allowed to do so. This is where client authentication comes in.
2. **Client Credentials Flow:** When a client requests an access token directly using its own credentials (client ID and secret), often for machine-to-machine communication.
 1. **Example:** The weather app directly requests an access token from the server using its own client ID and secret (like a username and password for the app). This is common for server-to-server communication, where no user is directly involved.
3. **Token Introspection:** When verifying the validity of an access token, client authentication ensures the request is from a legitimate source.
 1. **Example:** The shopping app sends the token to the authorization server to verify it's still active. The server checks if the request is from a legitimate app before confirming the token's validity.

How It Works:

Here we talking about the OAuth2 Client Authentication. Here we will see how to authenticate our application with the help of Google, Facebook, GitHub and many more such clients with the help of Spring Security. Here we are going to take a help of Google and similar process can be followed for GitHub, Facebook etc if you want so. Let's understand how it work.

Add Dependency

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Configure OAuth2

Add the dependency:

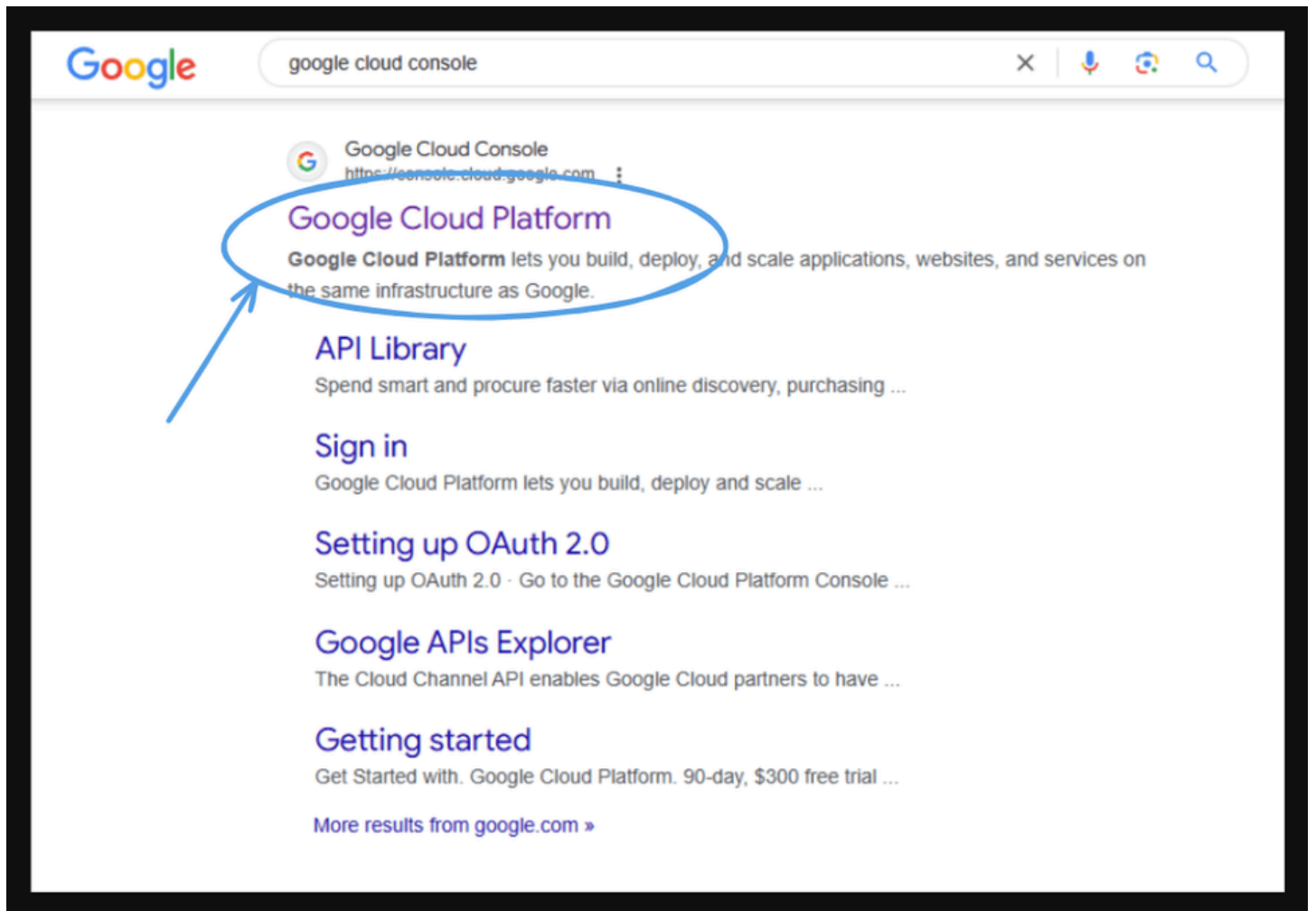
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Configure the WebSecurityConfig

```
.oauth2Login(oauth2Login ->
    oauth2Login
        .failureUrl("/login?error=true")
        .successHandler(oAuth2SuccessHandler)
    )
```

Google Cloud Console

- Go to the Google Cloud Console.



Google Cloud Console

And you will first have to create your Google developer account if you don't have one.

- Create your own project.

If you already have a project, go to 'My First Project' and select it.